Project 2: Efficient Dense Matrix Multiplication Report

1. How to Compile and Execute

The project is compiled and executed according to the `README.md`.

Compilation

```
# Navigate to the project directory
cd /path/to/project2

# Create a build directory and enter it
mkdir build && cd build

# Generate build files with CMake in Release mode
cmake ..

# Compile the project using 8 cores
make -j8
```

Execution

The programs are run on the cluster using the provided sbatch scripts to test the 1024x1024 (matrices 5x6) and 2048x2048 (matrices 7x8) test cases.

```
# Navigate to the project directory
cd /path/to/project2

# Submit the job script for matrices 5x6
sbatch ./src/scripts/sbatch_matmul5x6.sh

# Submit the job script for matrices 7x8
sbatch ./src/scripts/sbatch_matmul7x8.sh
```

2. Optimization Techniques and Parallel Models

This project systematically optimized a naive matrix multiplication implementation using a series of techniques covered in the course.

- **Compiler Optimizations (Task 1-2)**: These tasks focused on making the code more "compiler-friendly."
  - **FMA (Fused Multiply-Add)**: By using `__restrict__` pointers or standalone temporary variables, we signal to the compiler that memory regions do not overlap, allowing it to merge separate multiply and add instructions into a single, faster FMA instruction.

  - **Constant Pointers**: Declaring pointers to the input matrices as `const` provides further guarantees to the compiler, enabling more aggressive optimizations by confirming the data is read-only.

- **Memory Locality (Task 3)**: The naive `i-j-k` loop order results in poor spatial locality, as accessing `matrix2(k, j)` strides through memory, causing a cache miss on almost every access.
  - **Transposition**: This technique first create a transpose of `matrix2`. The multiplication then becomes `matrix1(i, k) * matrix2_T(j, k)`, which allows for contiguous memory access in the inner loop.
  - **Loop Interchange**: This "free" optimization reorders the loops (e.g., to `i-k-j` or `k-i-j`). This implementation used a cache-friendly ordering.

- **Tiling / Blocking (Task 4)**: By breaking the large matrices into smaller `BLOCK_SIZE` x `BLOCK_SIZE` sub-matrices, we can perform operations on blocks that are small enough to fit entirely within the L1 or L2 cache, minimizing cache capacity misses and data eviction.

- **Data-Level Parallelism (DLP) (Task 5)**: **GCC Auto-Vectorization** was enabled using `#pragma` directives. This instructs the compiler to use SIMD instructions, allowing a single CPU instruction to perform the same operation on multiple data elements simultaneously.

- **Thread-Level Parallelism (TLP) (Task 6)**: **OpenMP** was used to parallelize the outer loops of the tiled matrix multiplication. This is a shared-memory TLP model where multiple CPU cores work on different blocks of the result matrix concurrently, dramatically speeding up the computation.

- **GPU Acceleration (Extra Credit)**: **CUDA** was implemented as an extra credit task. This offloads the entire computation to the GPU, which executes thousands of threads in parallel to compute the result.

3. Experimental Results & Analysis

Performance was measured for all optimization steps on 1024x1024 and 2048x2048 matrices.

1024x1024 Matrix (matrices5x6)

Optimization Step	`BLOCK_SIZE`	Cores	Time (ms)	Speedup (vs. Naive)
Naive (Baseline)	-	1	12509	1.00x
Task 1.1: FMA (restrict)	-	1	8706	1.44x
Task 1.2: FMA (standalone)	-	1	8661	1.44x

Optimization Step	`BLOCK_SIZE`	Cores	Time (ms)	Speedup (vs. Naive)
Task 2: Const Ptr	-	1	3771	3.32x
Task 3.1: Transposition	-	1	1475	8.48x
Task 3.2: Loop Interchange	-	1	668	18.73x
Task 4.1: Tiling + Transpose	16	1	794	15.75x
Task 4.2: Tiling + Loop Interchange	64	1	612	20.44x
Task 5: Auto-Vectorization	32	1	321	38.97x
Task 6: OpenMP	32	32	42	297.83x
Bonus: CUDA	-	GPU	22.98	544.34x

2048x2048 Matrix (matrices7x8)

Optimization Step	`BLOCK_SIZE`	Cores	Time (ms)	Speedup (vs. Naive)
Naive (Baseline)	-	1	248167	1.00x
Task 1.1: FMA (restrict)	-	1	179221	1.38x
Task 1.2: FMA (standalone)	-	1	182244	1.36x
Task 2: Const Ptr	-	1	66762	3.72x
Task 3.1: Transposition	-	1	12473	19.90x
Task 3.2: Loop Interchange	-	1	7179	34.57x
Task 4.1: Tiling + Transpose	16	1	6392	38.82x
Task 4.2: Tiling + Loop Interchange	64	1	5002	49.61x
Task 5: Auto-Vectorization	32	1	2870	86.47x
Task 6: OpenMP	32	32	272	912.38x
Bonus: CUDA	-	GPU	147.51	1682.37x

Analysis

1. **Compiler Optimizations (Tasks 1-2)**: Simply enabling FMA and using `const` pointers gave a significant **3.3x-3.7x** speedup. This demonstrates that the initial naive code was severely limited by instruction dependencies and compiler ambiguity.
2. **Memory Locality (Tasks 3-4)**: This was the most important optimization. Fixing the strided memory access of the naive loop gave a massive performance boost.
    - **Loop Interchange** was far superior to **Transposition** (34.57x vs 19.90x speedup for 2048x2048), as it achieves locality without the $O(N^2)$ overhead of creating a new matrix.

- **Tiling** provided an additional modest, but consistent, improvement by enhancing temporal locality (e.g., 7179ms down to 5002ms for 2048x2048), proving the benefit of fitting blocks into cache. The best `BLOCK_SIZE` was consistently 16 or 64.

3. **Vectorization (Task 5)**: Applying auto-vectorization to the best tiled loop gave another **1.74x-1.90x** speedup. This shows that once the memory bottleneck was resolved, the code became compute-bound, and DLP (SIMD) was highly effective.
4. **Parallelism (Task 6 & Bonus)**:
   - **OpenMP** The 32-core version gave a **10.55x** speedup over the Auto-Vectorization version for the 2048x2048 case.
   - **CUDA** is in another class entirely. It is **1.84x** faster than the 32-core CPU version (147.5ms vs 272ms) and over **1600x** faster than the original naive code.

4. Profiling Analysis

`perf` Profiling (CPU)

The `perf` results for the 2048x2048 matrix clearly show the impact of our optimizations on hardware-level events.

Optimization Step (2048x2048)	`cpu-cycles` (Billions)	`cache-misses` (Billions)
Naive	708.0	5.170
FMA (restrict)	504.7	5.113
FMA (standalone)	511.2	5.328
Const Ptr	201.7	5.433
Transposition	45.8	0.668
Loop Interchange	29.7	0.669
Tiling + Transposition (BS=16)	28,2	0.535
Tiling + Loop Interchange (BS=64)	24.2	0.022
Auto-Vectorization (BS=32)	16.8	0.037
OpenMP (BS=32, 32 cores)	23.5	0.016

Analysis:

- **Tasks 1-2 (Compiler Opts)**: `cpu-cycles` dropped dramatically (708B to 201.7B) while `cache-misses` remained high. This confirms our bottleneck was initially instruction-level, not memory.
- **Task 3 (Loop Interchange)**: This was the turning point. `cache-misses` plummeted by 8.12x (5.4B to 0.67B). This directly proves that we solved the spatial locality problem.

- **Task 4 (Tiling)**: Tiling provided the final memory optimization, reducing `cache-misses` by another 30.4x (0.67B to 0.022B for BS=64). This confirms the effectiveness of blocking for temporal locality.
- **Tasks 5-6 (Parallelism)**: With cache misses minimized, `cpu-cycles` became the main bottleneck. Auto-vectorization reduced cycles further (24.2B to 16.8B). OpenMP used *more* total cycles (23.5B) but distributed them across 32 cores, resulting in the lowest wall-clock time.

`nsys` Profiling (CUDA)

The `nsys` profiler for the CUDA runs reveals the breakdown of GPU execution.

CUDA Run: 1024x1024 (matrices5x6)

- **Kernel Time**: 17,325,339 ns (**17.3 ms**)
- **API Overhead (`cudaMalloc`)**: 175,783,961 ns (**175.8 ms**)
- **Memcpy Time (HtoD + DtoH)**: 1,817,142 ns + 1,734,422 ns $\approx$ **3.55 ms**

CUDA Run: 2048x2048 (matrices7x8)

- **Kernel Time**: 137,276,961 ns (**137.3 ms**)
- **API Overhead (`cudaMalloc`)**: 136,852,297 ns (**136.9 ms**)
- **Memcpy Time (HtoD + DtoH)**: 15,850,666 ns + 7,901,157 ns $\approx$ **23.75 ms**

Analysis:

- **Kernel Scaling**: The computation is 8$\times$ larger for the 2048 matrix ($O(N^3)$). The kernel time scaled almost perfectly: 137.3 ms/17.3 ms $\approx$ 7.94x. This shows the GPU kernel itself is extremely efficient and scales predictably.
- **API Overhead**: The `cudaMalloc` call itself is a major, and highly variable, bottleneck, taking over 130ms in both cases.

5. Findings and Conclusion

This project was a clear demonstration of a systematic optimization process.

1. **Compiler Awareness is "Free" Performance**: Simply writing code that the compiler can understand and optimize (Tasks 1-2) yielded a 3-4x speedup.
2. **Memory is the Bottleneck**: The single most important optimization was fixing the memory access pattern (Task 3). The `perf` data confirmed a >8x reduction in cache misses (from 5.4B to 0.67B) , which directly translated to a >8x speedup over the compiler-optimized version.
3. **Hierarchy Matters**: Tiling (Task 4) further improved performance by respecting the size of the caches (L1/L2), not just the cache-line size. This was proven by `perf` showing another 30x drop in cache misses (from 0.67B to 0.022B).

4. **Parallelism is the Final Step**: Only after the memory bottlenecks were solved did parallelism (Tasks 5-6) become effective. Both data-level (SIMD) and thread-level (OpenMP) parallelism provided significant speedups.
5. **GPUs are Specialized for this Work**: The CUDA implementation was trivially the fastest, as its architecture is purpose-built for this kind of massively parallel, high-arithmetic-intensity task. Its performance scales almost perfectly with the $O(N^3)$ computational complexity, as shown by the kernel times , though it is still bound by API overhead and the cost of moving data across the PCIe bus.